

MIPS：入手与「优质」实践

MIPS: A Taste & Good Practice

陈康冰 邓飞 贾泽琛 谢东桓

零. 引言

零. 引言

前置需求

- Verilog HDL
- MIPS 架构与汇编
- Xilinx Vivado IDE
- MARS
- Microsoft Excel ☺
- 百度 / Bing / Google / GoogleScholar / Wikipedia / Github / StackOverflow

- **一阶不知道**：不知道某事——可以通过信息检索和学习获得相关的内容
- **二阶不知道**：不知道自己不知道某事——无法轻易获得相关内容
- **现状**：大部分的项目困难和问题都来自**二阶不知道**
- **愿望**：尽量将项目开发过程中的**二阶不知道**加以解决，以供参考

- 🎯 **目标**：在完成项目时需要加以考虑并解决的问题
- 🏆 **挑战**：在有余力的情况下可以进一步深入的内容，不必须，可能需要付出一定精力进行研究
- ✅ **优质实践**：在实际开发过程中的一些窍门或范例，不一定最优，但能起到一定参考作用
- 📖 **成果**：在进行到这一步时，已经实现的内容成果

零. 引言

内容安排

说明：

- 内容中的**加粗**或下划线词句，以及示意图中的词句，均为可供检索的关键词
- 内容中的删除线部分，表示不推荐，仅列出以供参考

一. 工具链和语言

- Verilog
- MARS
- Vivado

一. 工具链和语言

Vivado

工具链搭建与配置

| 目标

- 参见课程参考资源

简单的项目开发 workflow

| 目标



- 实践环节会进行演示

一. 工具链和语言

Vivado

源代码版本控制与协作

🏆 挑战

- Git & 在线 Git 托管服务 (Github、码云、GitLab 等)

外部编辑器

🏆 挑战

- Visual Studio Code

高级硬件描述语言和框架

🏆 挑战

- Chisel、SpinalHDL

一. 工具链和语言

Vivado

节约工具链时间的小技巧



优质实践

- 用于综合和实现：启用增量综合 (Incremental Synthesis) 以及增量实现 (Incremental Implementation)
- 在不必要时避免频繁修改存储器的初始化数据，因修改数据需要重新自定义存储器 IP 核，造成之前的综合和实现结果过期，需要重新综合，花费更多时间

一. 工具链和语言

MARS

简单 MIPS 汇编的编写



目标

- 和 x86 汇编编写思路的区别：频繁使用寄存器而非内存地址
- 参见课程提供的参考资料
- MARS 中自带了指令速查参考

一. 工具链和语言

MARS

简单测试程序开发 workflow



目标



一. 工具链和语言

MARS

MARS 的进阶使用与配置



挑战

- 延迟槽: Settings -> Delayed branching

一. 工具链和语言

MARS

MARSconv 小程序



优质实践

- https://gitee.com/ckblau/bitmips21_resource/blob/master/marsconv.c
- 自动实现从 MARS 导出的 16 进制格式到 Vivado \$readmemh 支持的格式的转换
- 适用于使用 \$readmemh 或 \$readmemb 载入机器码文件的情况

一. 工具链和语言

Verilog

简单 Verilog 代码的编写 |  目标

- 语法格式、多文件编程等

进阶 Verilog 语法 |  挑战

- 函数、循环、头文件和宏定义等

一. 工具链和语言

Verilog

命名和语法规范



优质实践

- 顶层信号命名：帕斯卡命名法 [名称]+[所属级(F/D/E/M/W)]
 - 例：ALUCtrlD, WriteRegE
- 元件接口信号：下划线命名法 全小写
 - 例：d_addr, num_in1
- 缩进与对齐：信号声明、长表达式、元件接口定义、元件实例化

一. 工具链和语言

Verilog

命名和语法规范

✓ 优质实践

```
787
788     assign HiInE = HiLoWriteE == 2'b10 ? WriteCP0HiLoDataE : HiResultE;
789     assign LoInE = HiLoWriteE == 2'b01 ? WriteCP0HiLoDataE : LoResultE;
790     hilo_hilo(
791         .clk(clk), .rst(rst),
792         // input
793         .we          (HiLoWriteW),
794         .hi_i         (HiInW),
795         .lo_i         (LoInW),
796         // output
797         .hi_o         (HiOutE),
798         .lo_o         (LoOutE)
799     );
800     assign HiLoOutE = sel_2to4(HiLoToRegE,
801                               32'b0,
802                               sel_2to4(ForwardHE, LoOutE, LoInW, LoInM, 32'b0),
803                               sel_2to4(ForwardHE, HiOutE, HiInW, HiInM, 32'b0),
804                               32'b0);
805
806     assign ExceptionTypeE = ExceptionTypeE_in ? ExceptionTypeE_in :
807                               ({32{DataUnalignedE && MemToRegE}} & `EXCEP_MASK_AdELD) |
808                               ({32{DataUnalignedE && MemWriteE}} & `EXCEP_MASK_AdES) |
809                               ({32{OverflowE}} & `EXCEP_MASK_Ov);
810
```

一. 工具链和语言

Verilog

一些较好的 Verilog 实现 | ☒ 优质实践

- 位扩展：
 - `{16'b0, InstrD[15:0]}` // 无符号
 - `{{16{InstrD[15]}}, InstrD[15:0]}` // 带符号
- 选择器：
 - 二路选择器：三目运算符 `sel ? in1 : in2`
 - 四路选择器：见下页
- 算术与逻辑运算：
 - Verilog 支持常见的运算，以及 `{}` 运算，即信号合并运算

一. 工具链和语言

Verilog

一些较好的 Verilog 实现 | ☒ 优质实践

- 四路选择器:

- ```
function [31:0] sel_2to4(input [1:0] sel,
```
- ```
                        input [31:0] in0, input [31:0] in1,
```
- ```
 input [31:0] in2, input [31:0] in3);
```
- ```
begin
```
- ```
 case (sel)
```
- ```
        2'b00 : sel_2to4 = in0;
```
- ```
 2'b01 : sel_2to4 = in1;
```
- ```
        2'b10 : sel_2to4 = in2;
```
- ```
 2'b11 : sel_2to4 = in3;
```
- ```
    endcase
```
- ```
end
```
- ```
endfunction
```

一. 工具链和语言

Verilog

时序和组合逻辑的一些注意点 | 优质实践

- 单周期 CPU 内或流水线 CPU 同一流水级内的单个信号，不能连续经过多个带相同时钟沿触发的时序逻辑
- 简单的运算，如位扩展、加减法等，使用纯组合逻辑
- 寄存器读可以使用纯组合逻辑，但写必须使用时序逻辑
- 慎用 `always @(*)`
- 对于存在读写的元件，推荐上升沿触发读取，下降沿触发写入

一. 工具链和语言

成果

- 工具链的建立、配置和简单使用
- 项目的建立、编码、仿真等工作流过程
- Verilog 语言程序的读写能力

二. 单周期CPU设计

- 系统架构与接口设计
- 核心架构与元件实现
- 测试程序编写以及仿真调试
- 上板运行与在线调试

二. 单周期CPU设计

系统架构与接口设计

从系统架构拆分开始

|  目标

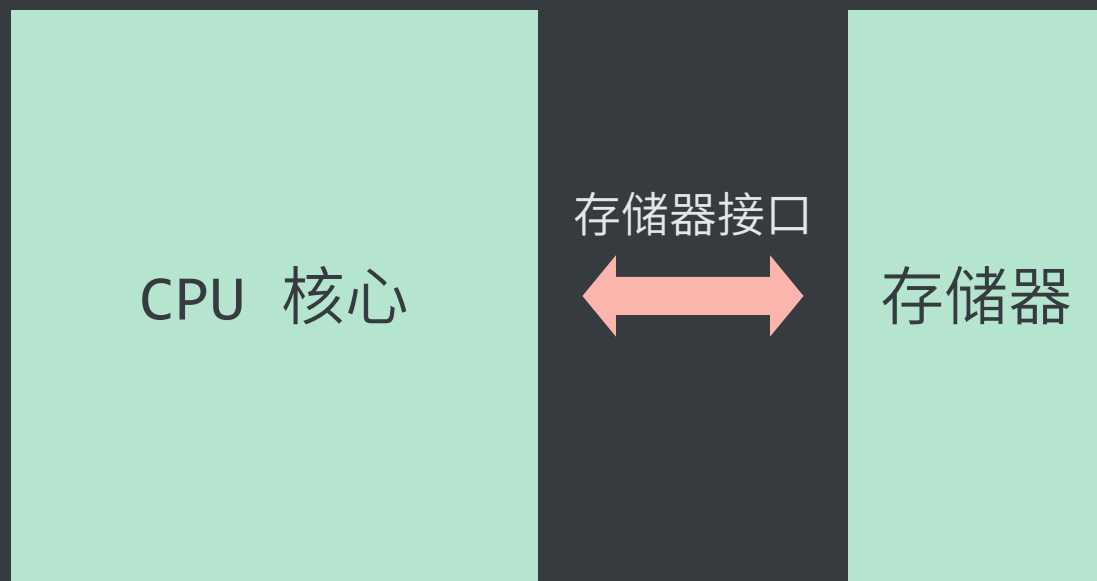
系统

二. 单周期CPU设计

系统架构与接口设计

从系统架构拆分开始

| 目标



二. 单周期CPU设计

系统架构与接口设计

从系统架构拆分开始

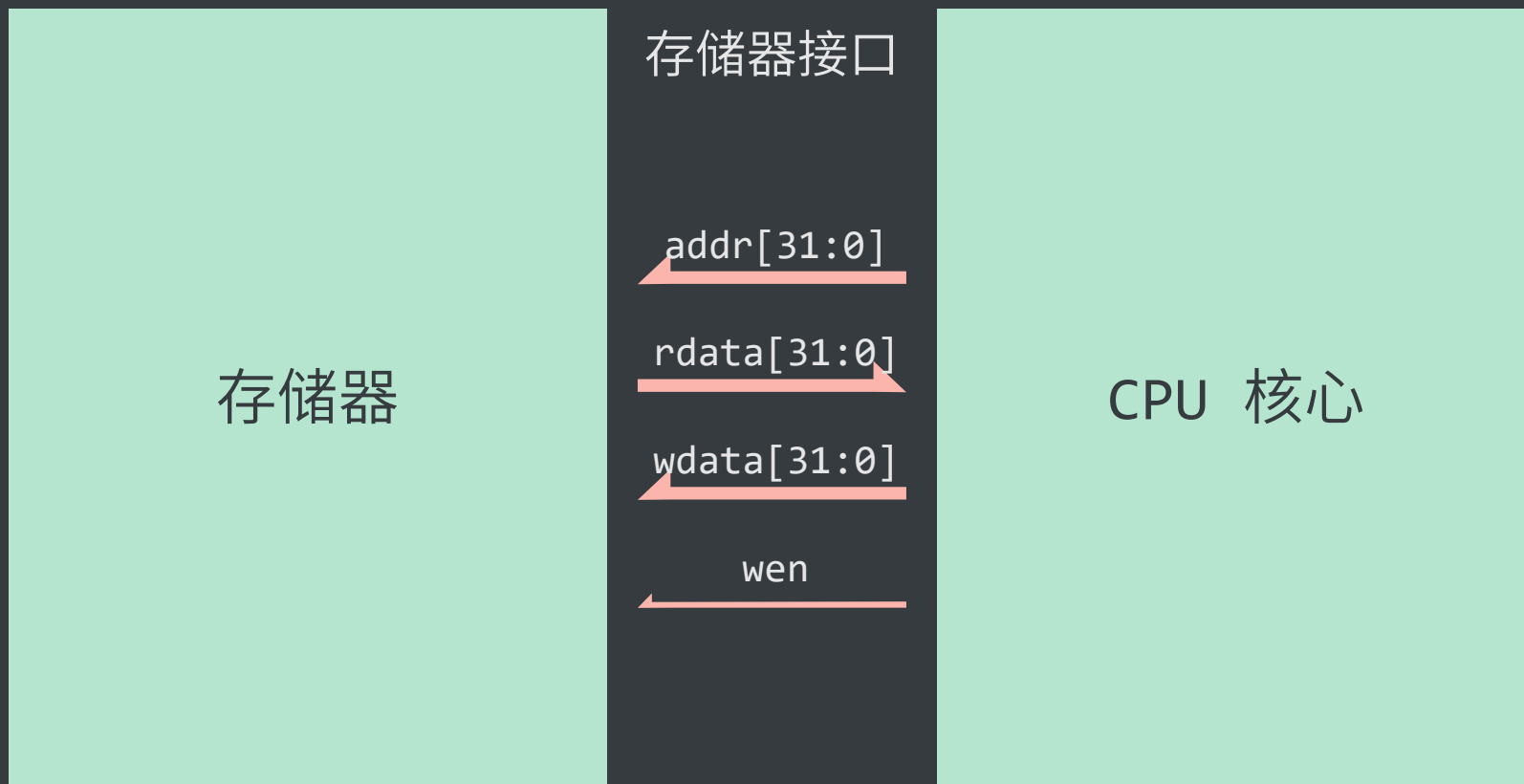
| 目标



二. 单周期CPU设计

系统架构与接口设计

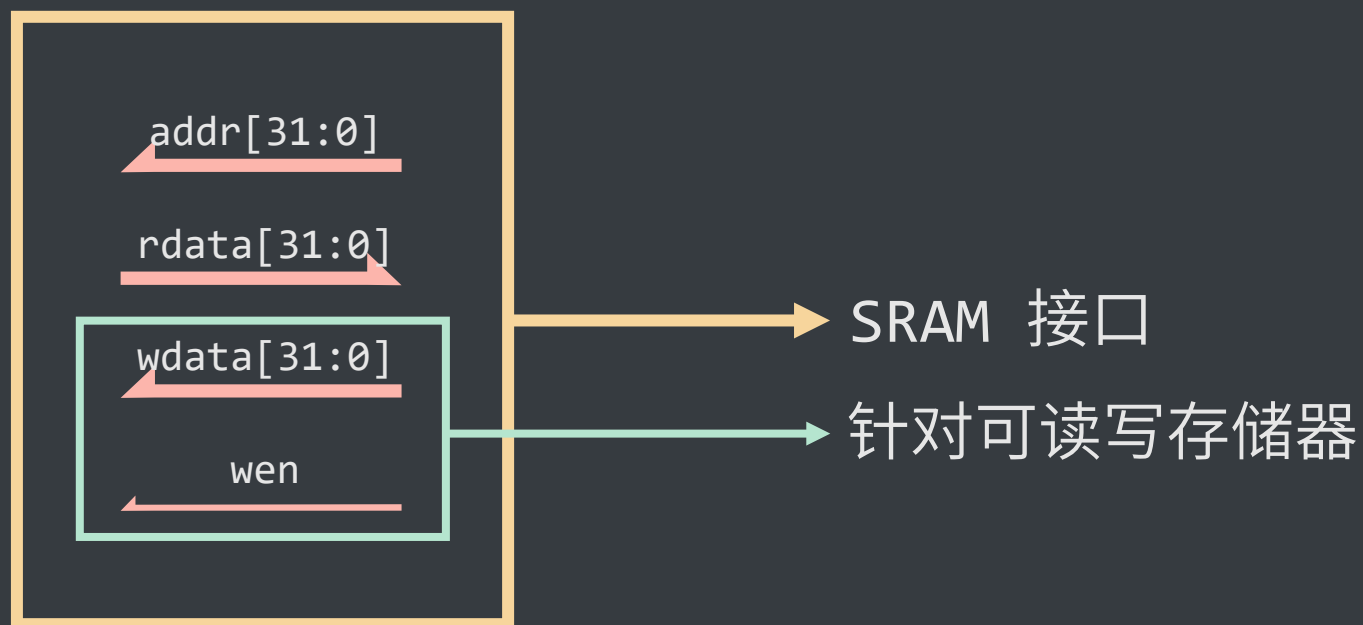
存储器接口设计



二. 单周期CPU设计

系统架构与接口设计

存储器接口设计



二. 单周期CPU设计

系统架构与接口设计

存储器接口设计

| 目标



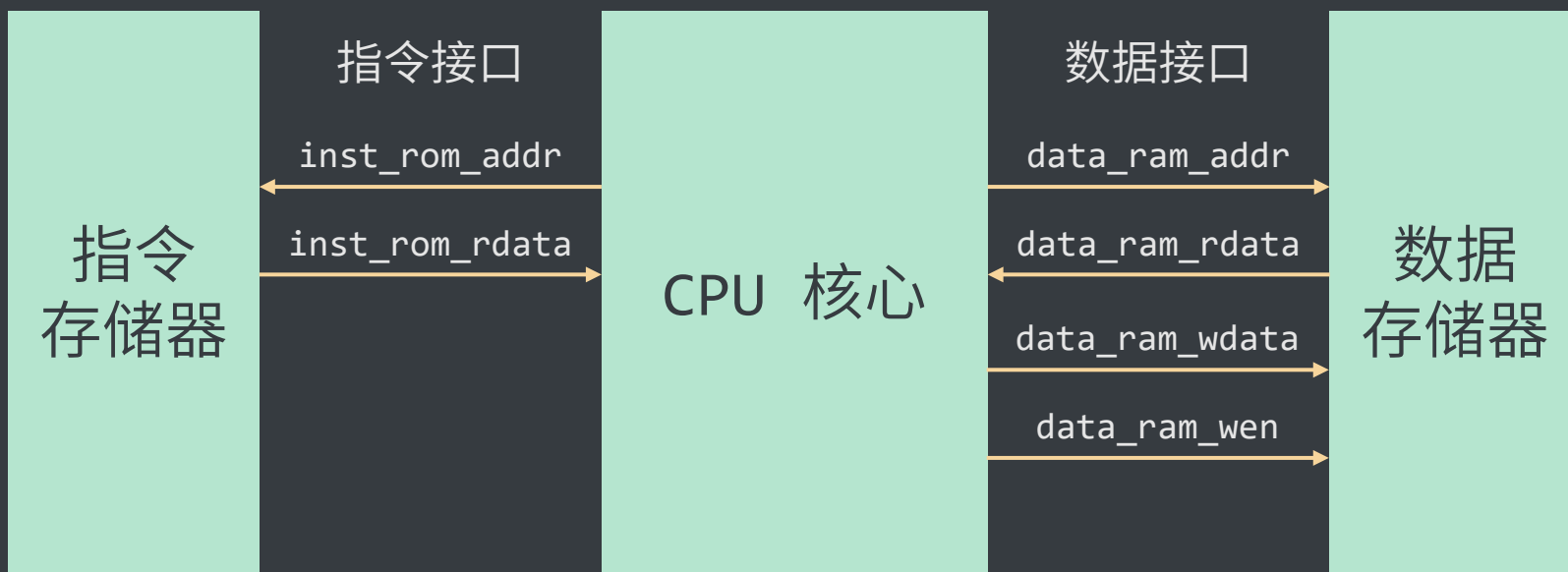
二. 单周期CPU设计

系统架构与接口设计

存储器接口设计



目标



https://github.com/bit-mips/bitmips_experiments_doc/blob/master/lab3/singlecycle.md%E6%8E%A5%E5%8F%A3%E5%AE%9A%E4%B9%89

二. 单周期CPU设计

系统架构与接口设计

存储器接口设计



```
22
23 module mycpu (
24     input      rstn,
25     input      clk,
26
27     output [31:0] inst_rom_addr,
28     input  [31:0] inst_rom_rdata,
29
30     output [31:0] data_ram_addr,
31     output [31:0] data_ram_wdata,
32     output      data_ram_wen,
33     input  [31:0] data_ram_rdata
34 );
35
```

存储器接口进阶设计

挑战

- ~~非哈佛架构访存设计：双口存储器~~
- 非 SRAM 接口访存设计：
 - 带握手信号的类 SRAM 接口
 - AXI 接口
- 不基于 reg 数组的存储器设计：Xilinx Block RAM IP 核

二. 单周期CPU设计

系统架构与接口设计

存储器模块设计与使用



目标



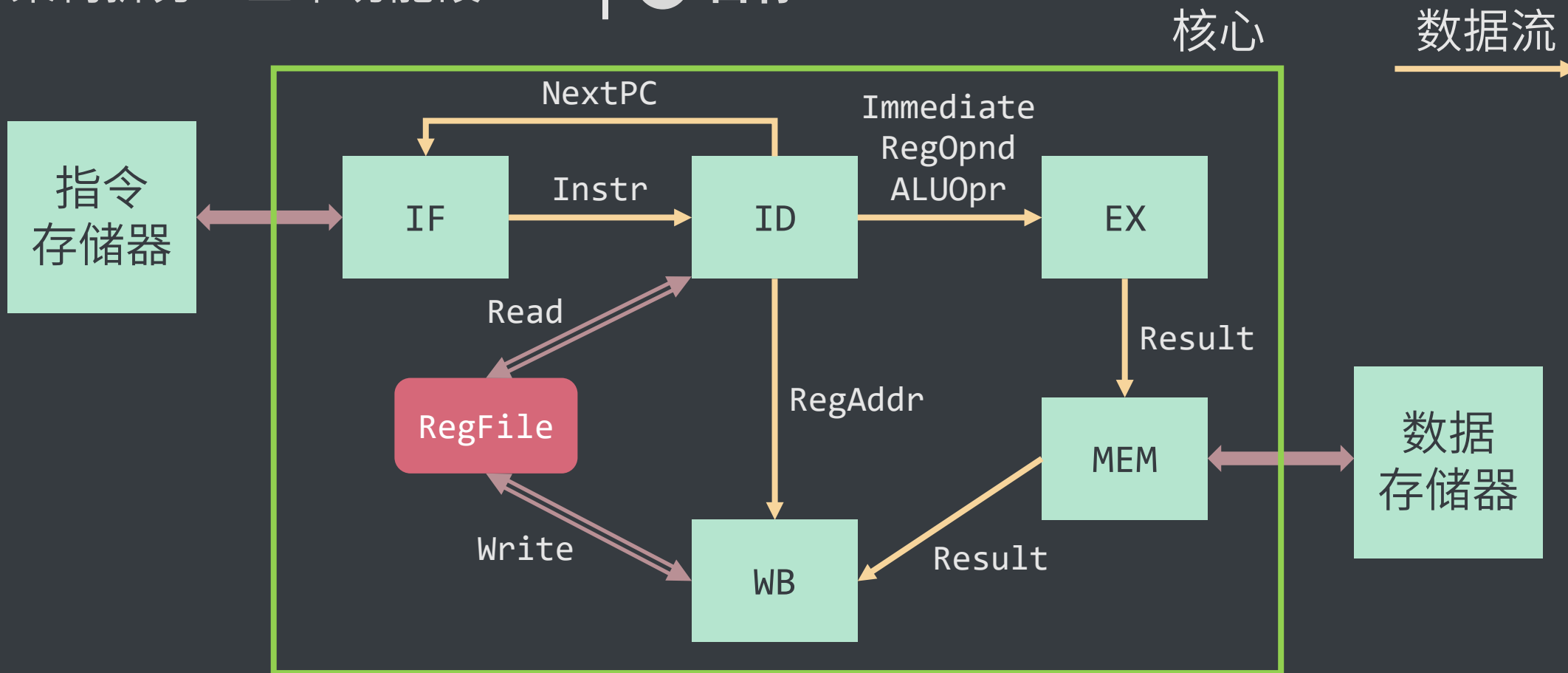
优质实践

- 存储器实现：参见提供的单周期代码中 `dmem` 模块和 `imem` 模块
- 数据载入：使用 `$readmemh` 载入 MARS 生成的测试程序机器码，具体操作会在实践流程中介绍
- 两个环境：
 - 自定义的 `testbench` 环境（使用 `reg` 实现存储器）
 - 带 `SoC` 的环境（使用 `SoC` 自带存储器，无需实现）

二. 单周期CPU设计

核心架构与元件实现

架构拆分：五个功能段



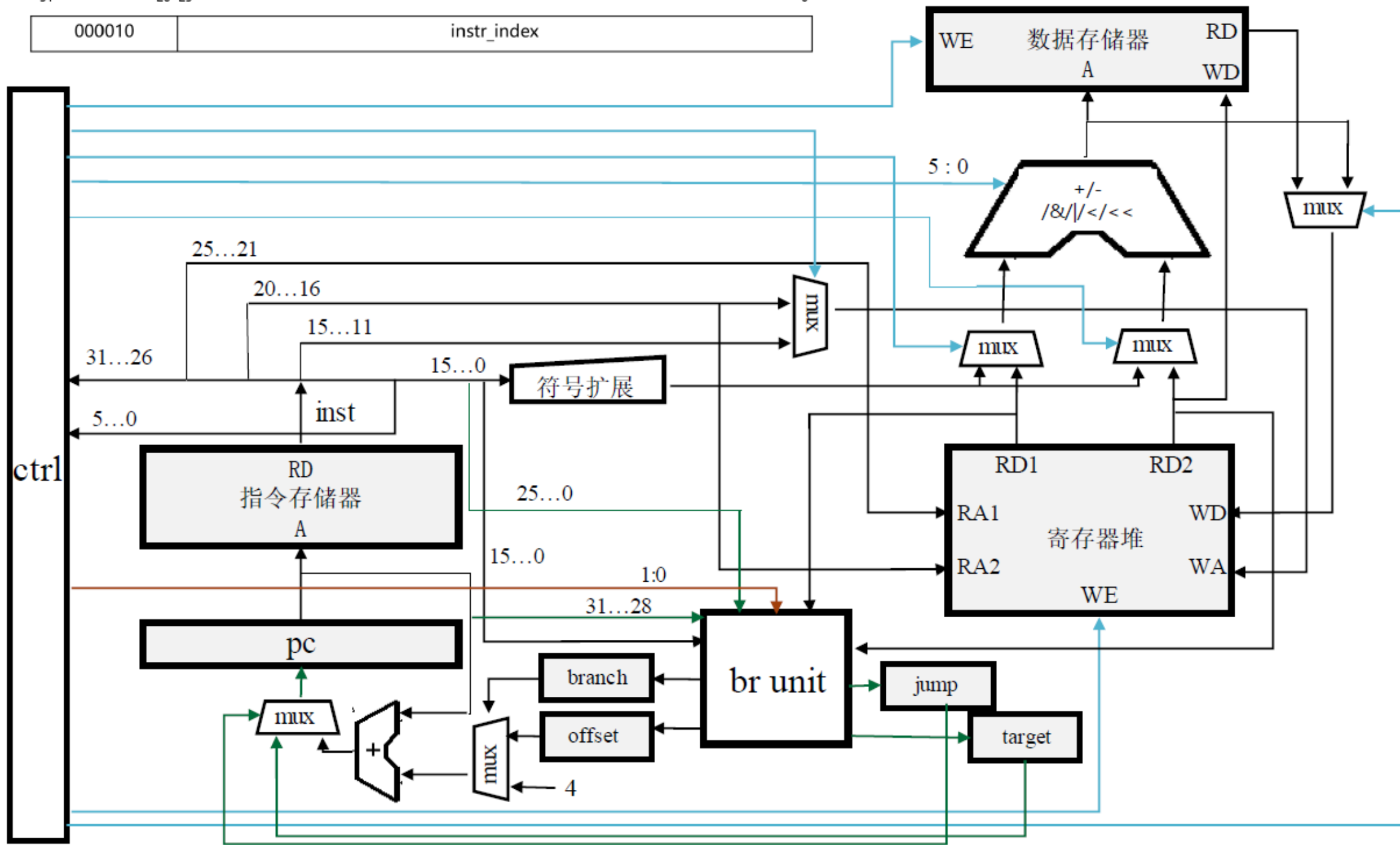
二. 单周期CPU设计

核心架构与元件实现

功能段内部实现

|  目标

- “课程 07: MIPS 单周期 CPU 设计”

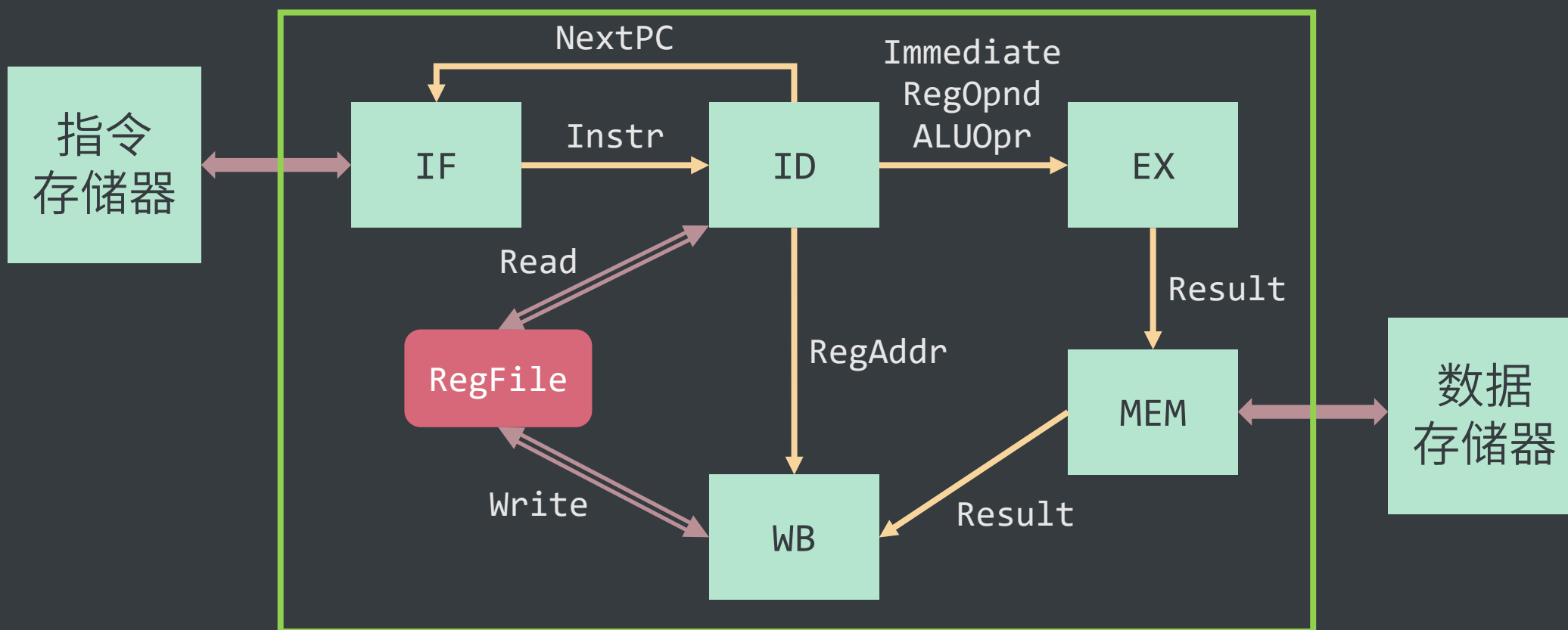


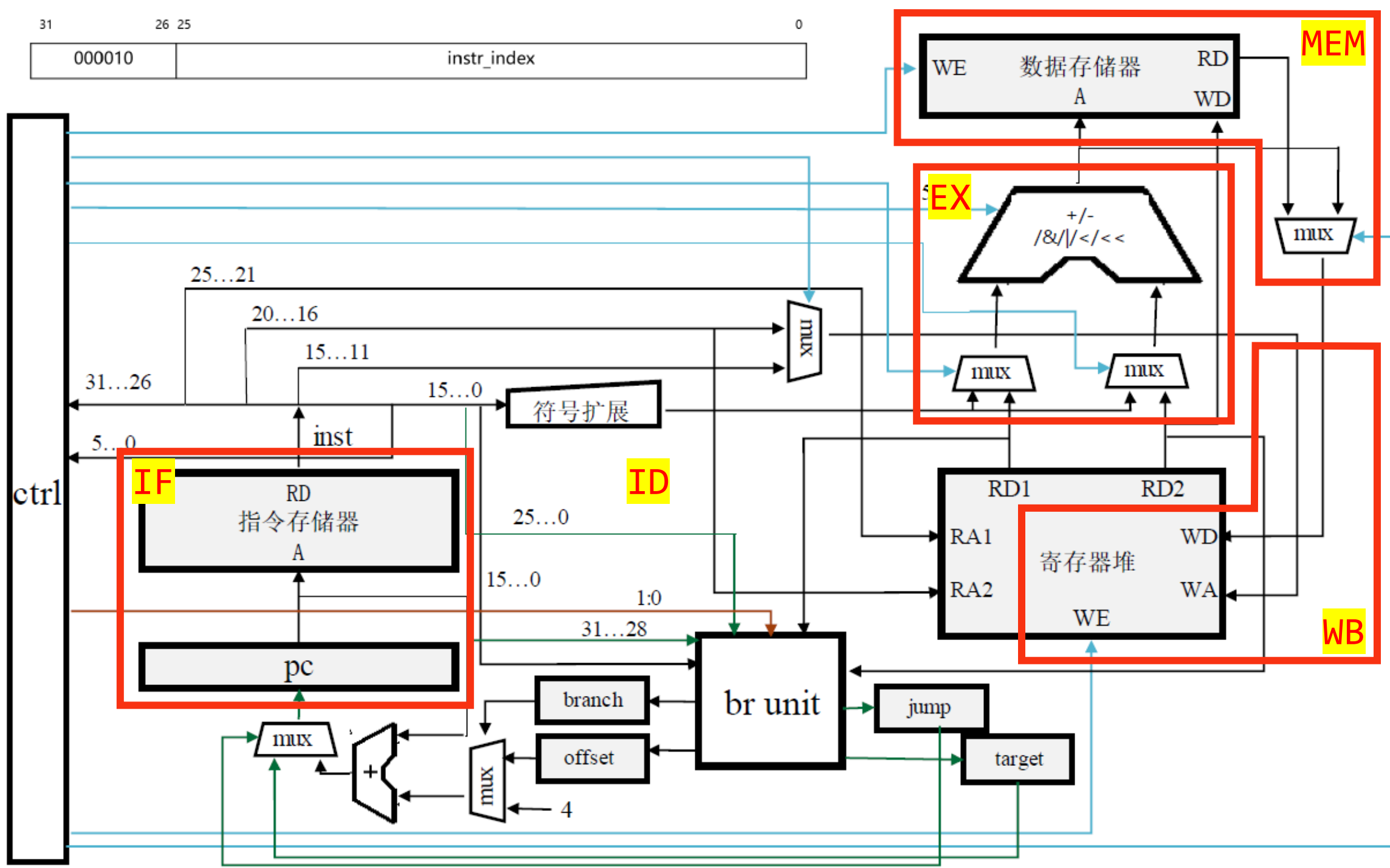
二. 单周期CPU设计

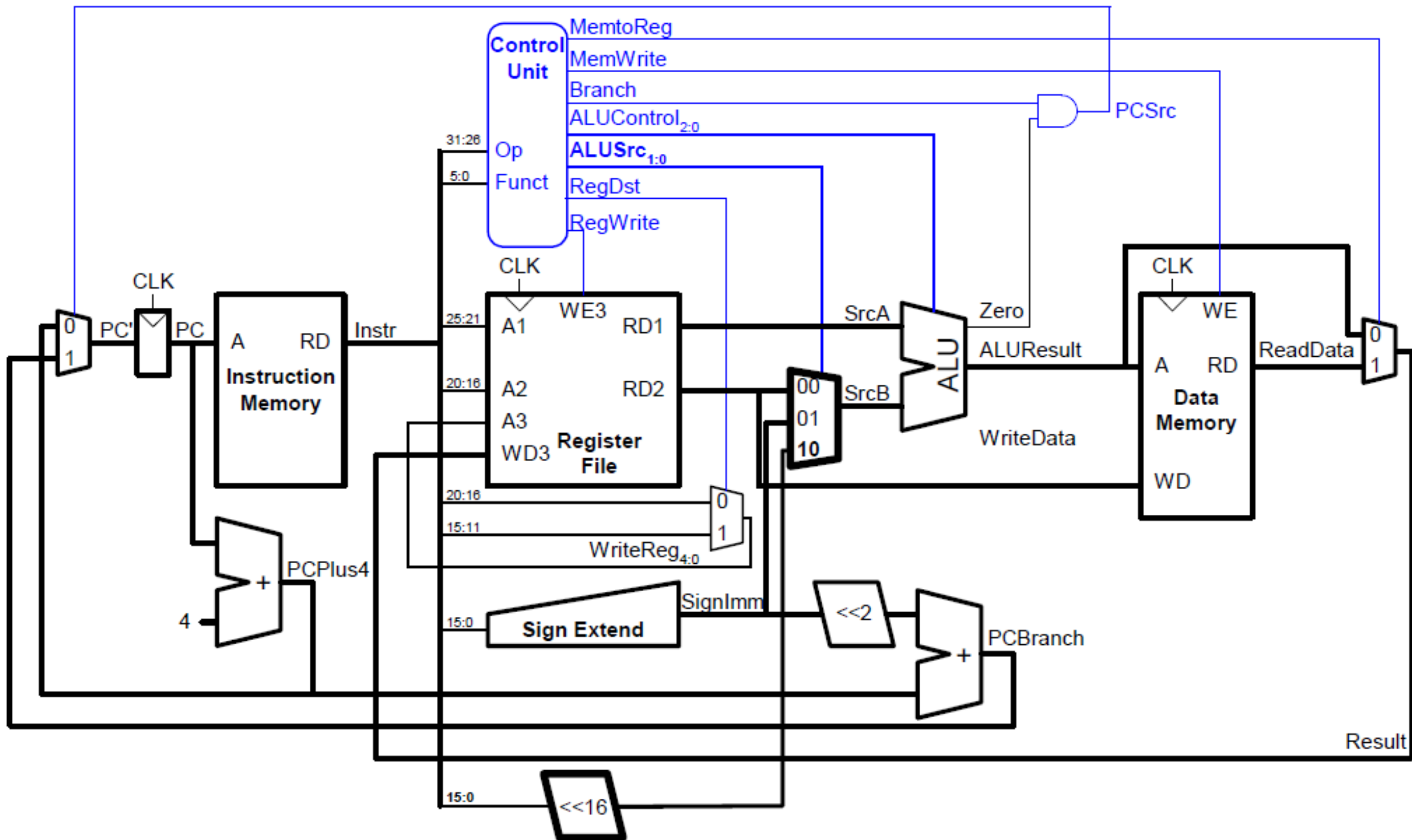
核心架构与元件实现

功能段内部实现

| 目标





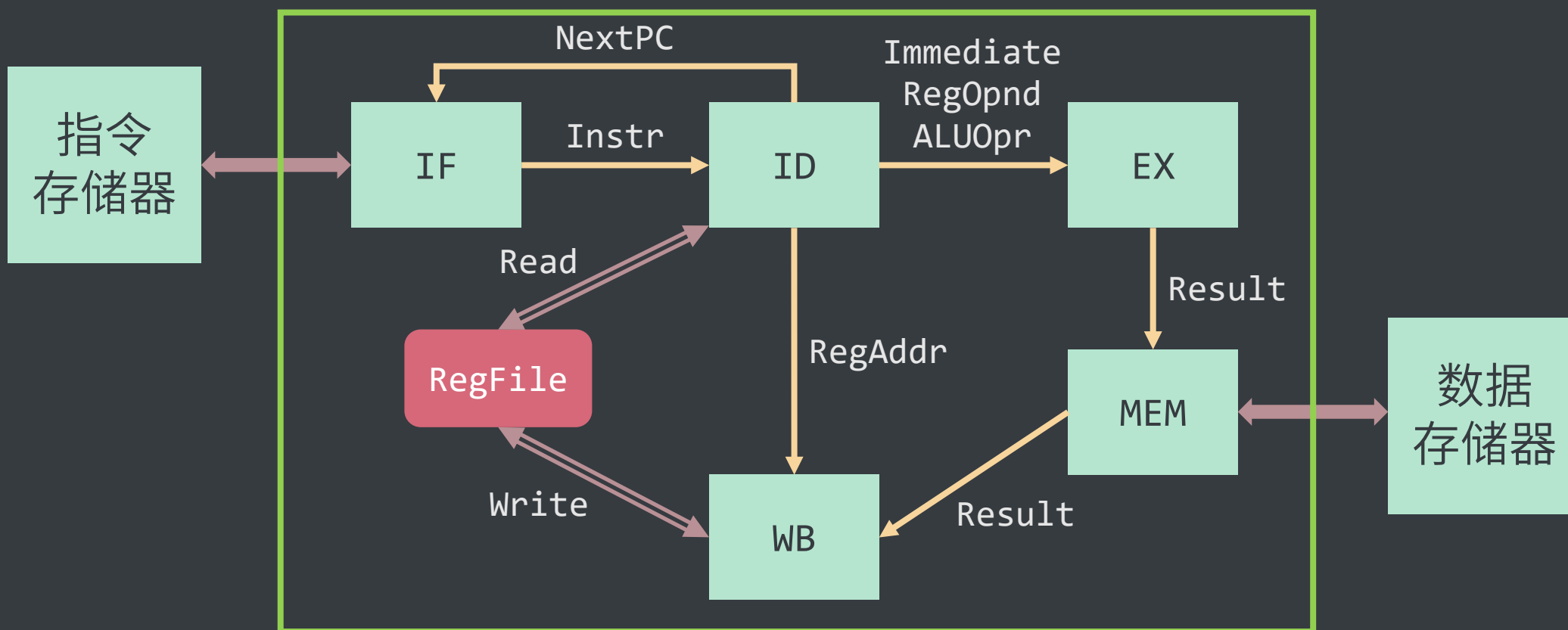


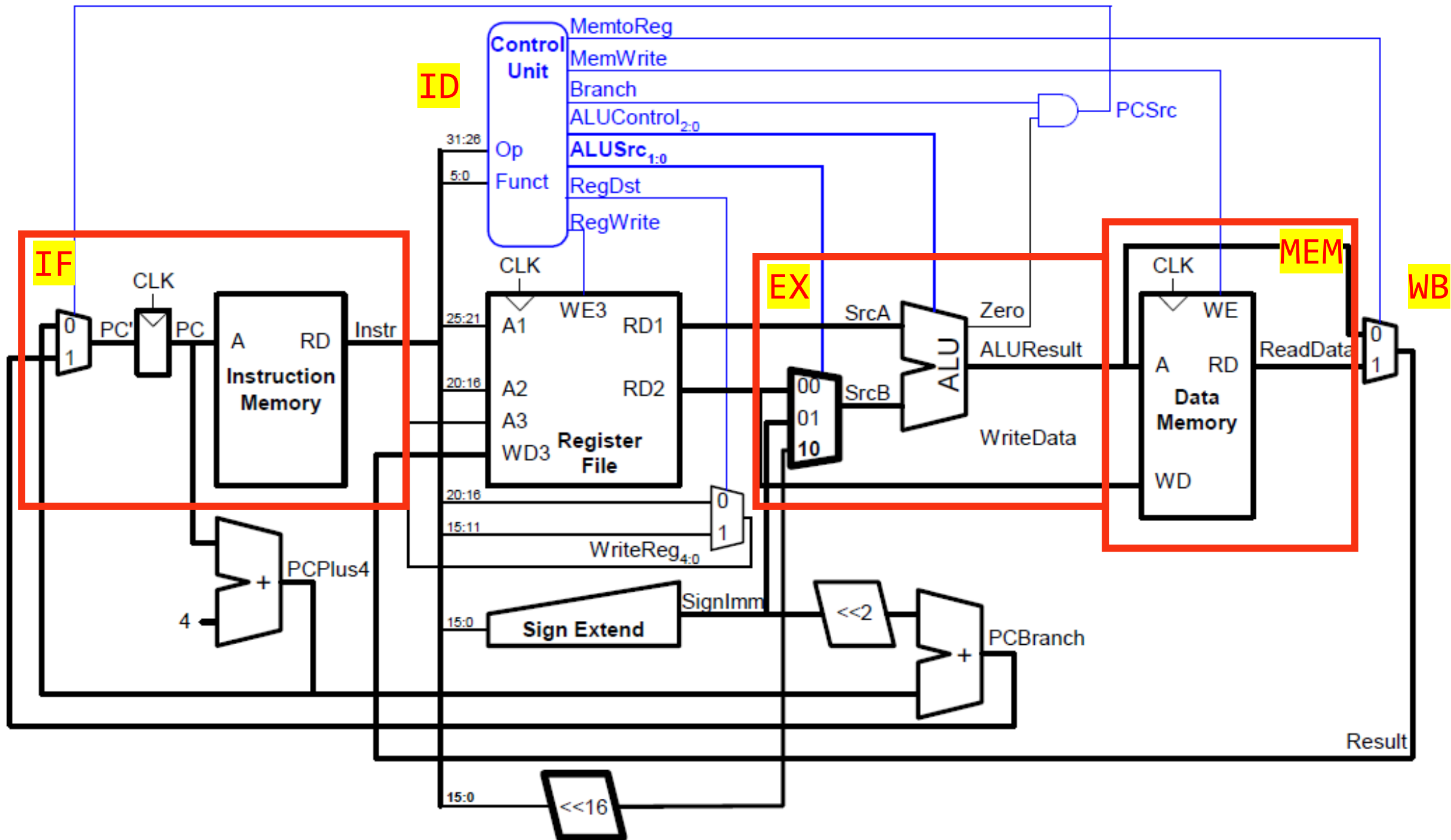
二. 单周期CPU设计

核心架构与元件实现

功能段内部实现

目标





二. 单周期CPU设计

核心架构与元件实现

元件实现

| 目标

- RegFile、PC、ALU、CU
- 参考：
 - 课程 07：MIPS 单周期 CPU 设计
 - 所提供的单周期 CPU 代码

二. 单周期CPU设计

核心架构与元件实现

元件的单元测试

挑战

- 元件实现完成后可以通过自己开发的 `testbench` 对元件功能进行测试，可以参考数字逻辑课程中为元件编写 `testbench` 的相关内容
- 有些问题（比如时序）可能无法通过元件本身的单元测试发现，故测试的重点还是核心的集成测试，单元测试回报不大，在对代码较为熟悉的情况下，静态差错也不失为一个好的选择

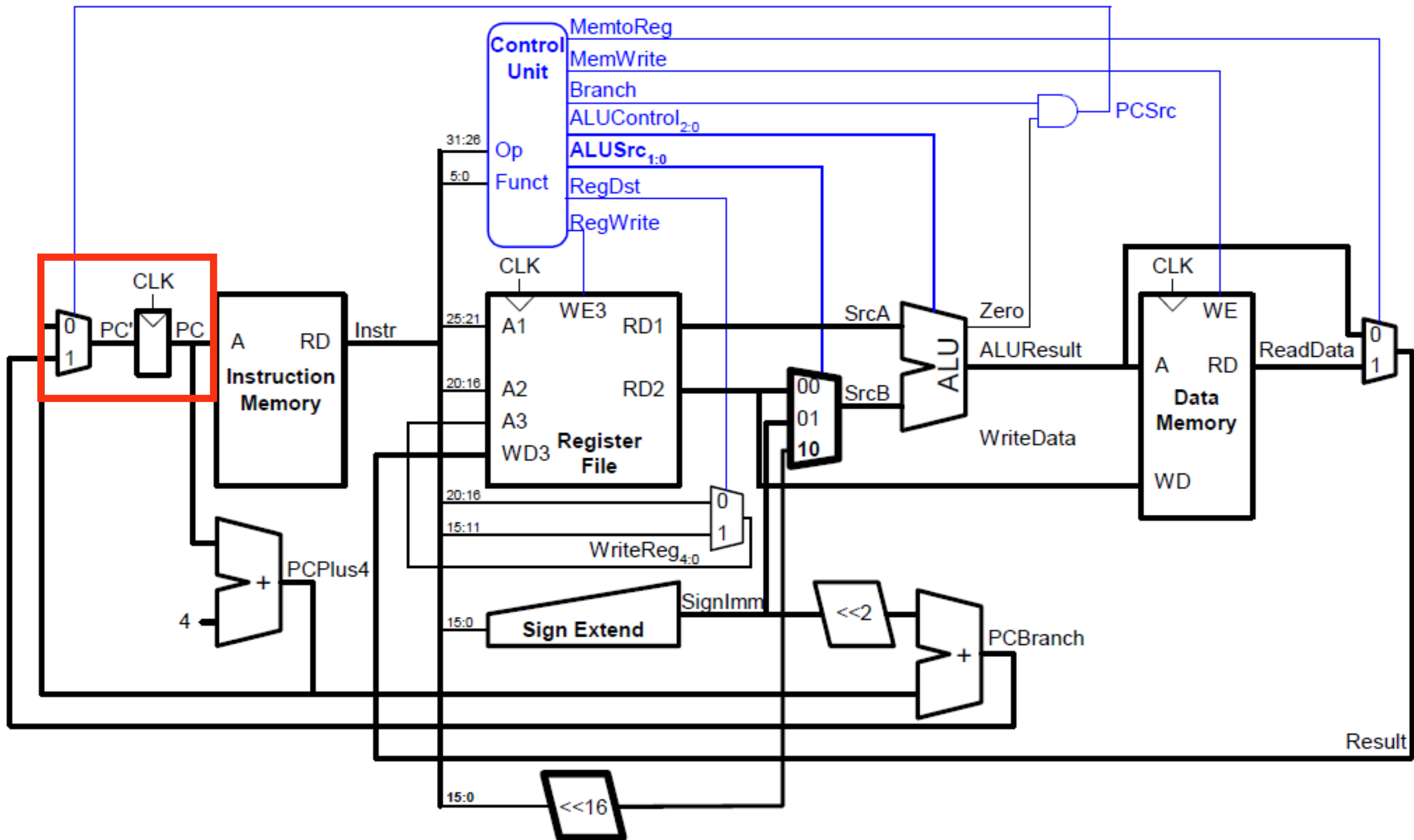
二. 单周期CPU设计

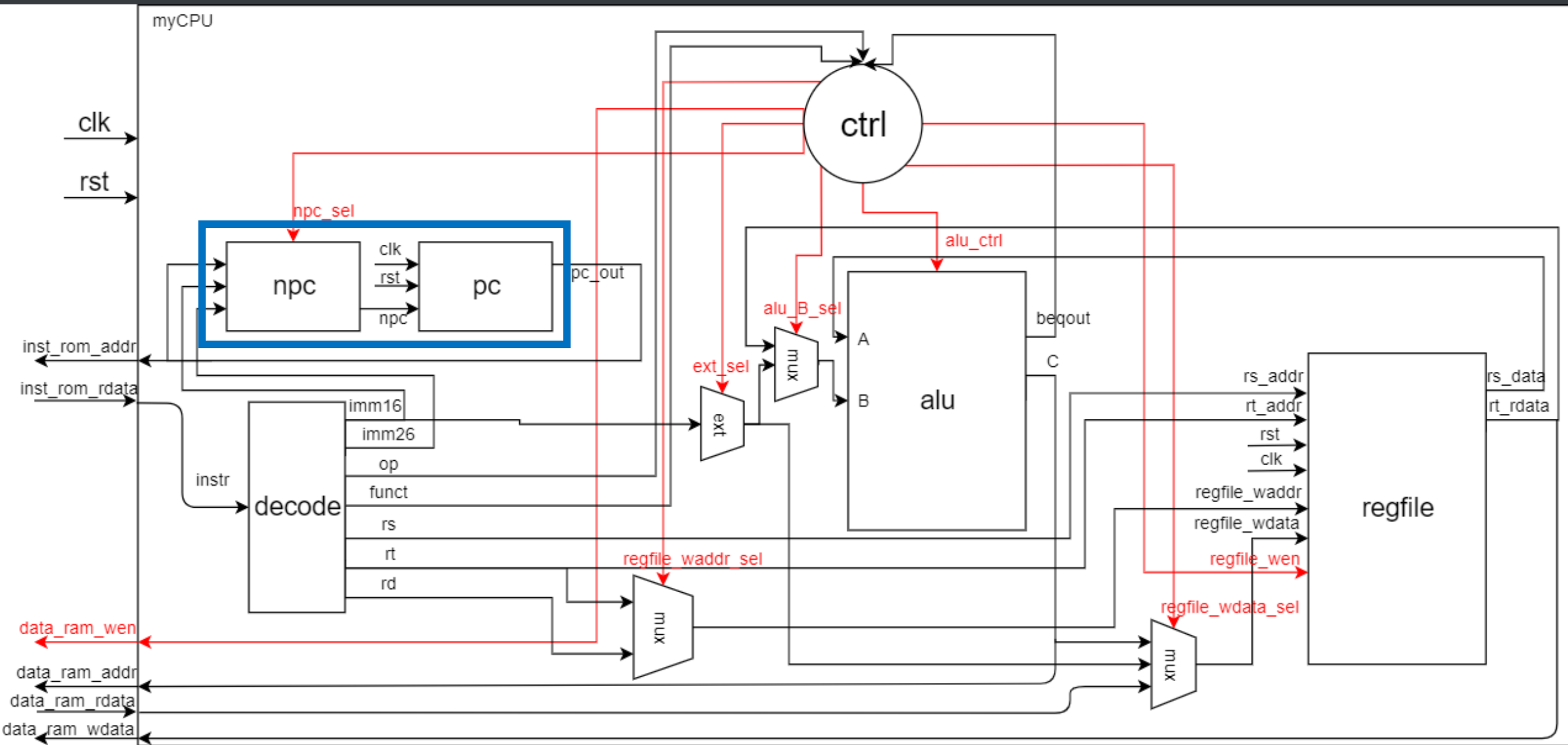
核心架构与元件实现

延迟槽机制

挑战

- 之前课程中展示的单周期架构并未实现延迟槽功能
- Harris 《计算机组成与设计：硬件软件接口》中的单周期架构设计也支持延迟槽功能
- 下面的链接中提供了另一个支持延迟槽功能的单周期架构设计
https://github.com/bit-mips/bitmips_experiments_doc/blob/master/lab3/singlecycle.md#%E6%8E%A5%E5%8F%A3%E5%AE%9A%E4%B9%89





二. 单周期CPU设计

核心架构与元件实现

添加更多指令

|  挑战

- 运算指令：
 - 添加 ALU 操作，实现其他算术和逻辑运算指令
 - 复用不同类型运算指令的控制和数据信号，支持 R-Type 和 I-Type 的运算指令
 - 添加 HiLo 寄存器和乘除法器，实现乘除法运算指令

二. 单周期CPU设计

核心架构与元件实现

添加更多指令

|  挑战

- 跳转分支指令：
 - 在 ID 段进行分支跳转判断和地址生成 (br unit)，添加不同的跳转条件，实现其他的跳转分支指令
 - 添加 PC 到寄存器的传输通路和相关控制信号，实现写返回地址寄存器的跳转指令
- 访存指令：
 - 添加访存位宽信号，实现不同宽度的访存指令

二. 单周期CPU设计

核心架构与元件实现

面向 Excel 编程 ☺



优质实践

- 指令集.xlsx

二. 单周期CPU设计

测试程序编写以及仿真调试

测试程序编写

| 目标

- 黑盒测试 or 白盒测试?
- 针对指令数据范围确定边界值，划分等价类
- 设计测试用例覆盖有效等价类和无效等价类
-

二. 单周期CPU设计

测试程序编写以及仿真调试

测试程序编写



目标



优质实践

- 运算指令（以 ADDU 为例）：
 - `lui t0, 0x1ade`
 - `ori t0, t0, 0xf300` `// 0x1adef300`
 - `lui t1, 0x51dd`
 - `ori t1, t1, 0x58de` `// 0x51dd58de`
 - `lui v1, 0x6cbc`
 - `ori v1, v1, 0x4bde` `// 0x6cbc4bde`
 - `addu v0, t0, t1`
 - `beq v0, v1, <error>`
 - `nop`
 - `j <correct>`
 - `nop`

二. 单周期CPU设计

测试程序编写以及仿真调试

测试程序编写



目标



优质实践

- 跳转指令（以 BNE 为例）：

- `lui t0, 0xbcaa`
- `ori t0, t0, 0xbd27` `// 0xbcaabd27`
- `lui t1, 0x82a7`
- `ori t1, t1, 0x7a9d` `// 0x82a77a9d`
- `bne t0, t1, <next>`
- `nop`
- `j <error>`
- `nop`
- `next:`
- `lui t0, 0xbcaa`
- `ori t0, t0, 0xbd27` `// 0xbcaabd27`
- `lui t1, 0xbcaa`
- `ori t1, t1, 0xbd27` `// 0xbcaabd27`
- `bne t0, t1, <error>`
- `nop`
- `j <correct>`
- `nop`

二. 单周期CPU设计

测试程序编写以及仿真调试

仿真调试



目标

- 参见“课程 01: Vivado 仿真介绍”
- 实践环节会进行演示

仿真设置



优质实践

- 在 Simulation Settings 中启用 `xsim.simulate.log_all_signals` 选项，即可在仿真时记录所有的信号，修改抓取的信号时即可省去重新运行仿真的时间

二. 单周期CPU设计

测试程序编写以及仿真调试

Trace 机制

挑战

- 对于较大的程序或者较复杂的 CPU 架构，在仿真中看波形进行调试会相当繁琐，而且难以定位问题，利用 trace 机制即可有效解决这一问题
- trace 机制指：实时抓取 CPU 的 PC 和寄存器写回情况，即写回的目标寄存器地址和写回值，通过对比当前的 PC、写回情况和正确的 PC、写回情况，即可判断当前 CPU 执行是否出现问题
- 以下链接给出了在开发板 SOC 环境中应用 trace 机制的教程
- https://github.com/bit-mips/bitmips_experiments_doc/blob/master/lab5/introduction.md
- https://github.com/bit-mips/bitmips_experiments_doc/blob/master/lab5/soc-interface.md

二. 单周期CPU设计

上板运行与在线调试

上板运行

挑战

- 参见“课程 09: Vivado 综合上板介绍”
- 实践环节会进行演示

在线调试

挑战

- 使用 Vivado ILA 功能进行上板调试
- 综合后 Setup Debug, 选择需要抓取的信号, 建立 Xilinx ILA 核
- 上板后连接 ILA 核, 设置触发条件

二. 单周期CPU设计

成果

- 支持 10 条基本指令的单周期 CPU
- 针对 CPU 10 条基本指令的功能测试程序
- CPU 的仿真测试与上板运行

三. 接口与外设

- 外设接口设计与实现
- 外设模块设计与实现
- 外设测试程序编写与仿真调试

三. 接口与外设

外设接口设计与实现

外设接口分析

| 目标

- 外设分类：输入设备、输出设备、输入输出设备
- 外设接口需要传输的内容：控制指令、数据，和访存类似
- 外设接口传输内容的形式：字节、字节流，和访存类似
- 外设接口传输内容的方向：和访存方向类似
- 外设接口选择特定外设的方法：给外设分配地址，和访存类似
- 综上，设计外设接口时可以借鉴访存接口的设计

三. 接口与外设

外设接口设计与实现

外设接口设计

| 目标

- 内存映射式接口：
 - 并联在内存总线上，响应针对特定地址的访存操作
 - 将访存的读取和写入操作映射为对外设的读写操作
- 外设专用接口：
 - 有专门的外设总线和外设访问指令（x86 IN OUT）
 - 机制和访存类似，但有更高自由度

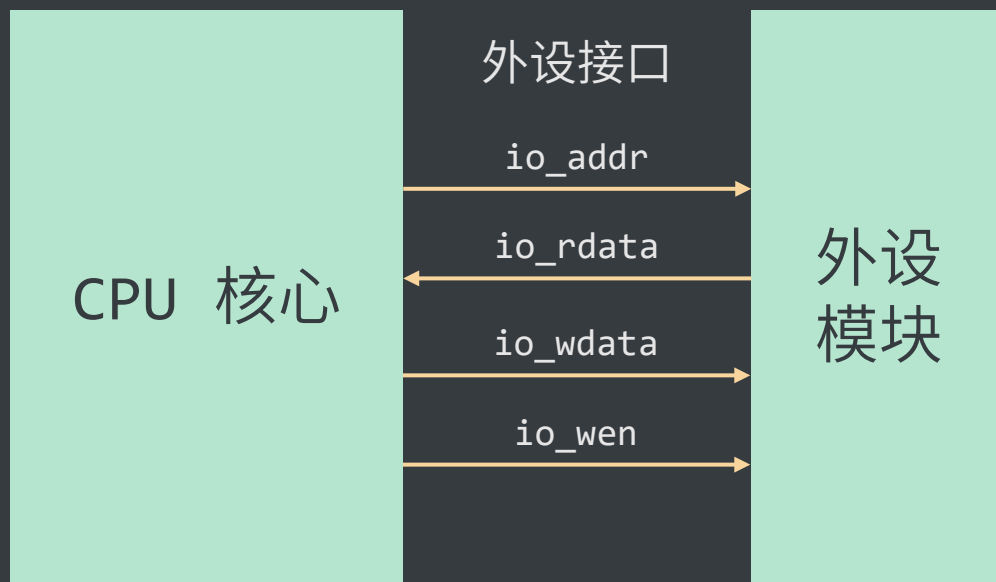
三. 接口与外设

外设模块设计与实现

外设模块抽象设计



目标

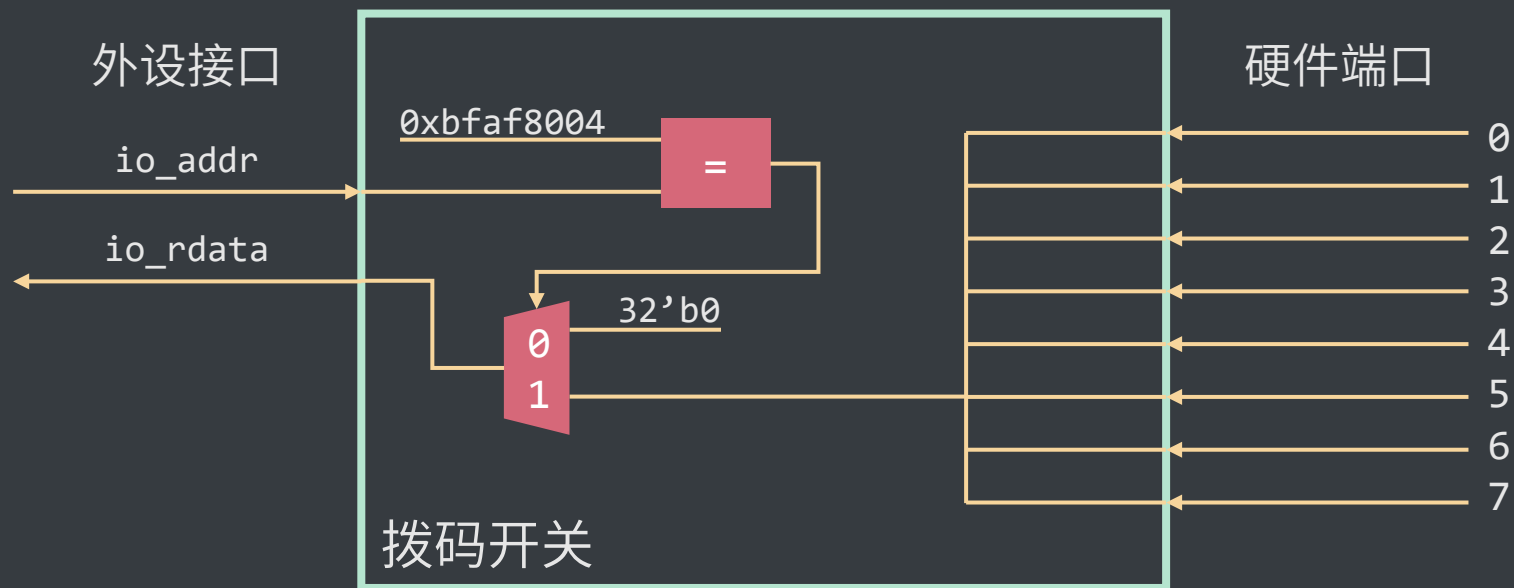


三. 接口与外设

外设模块设计与实现

外设模块内部实现

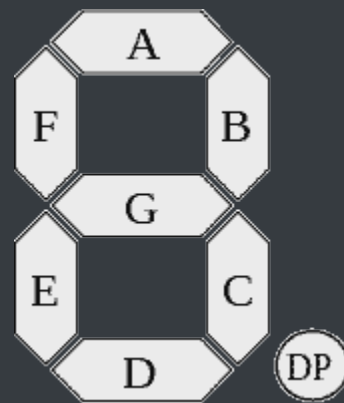
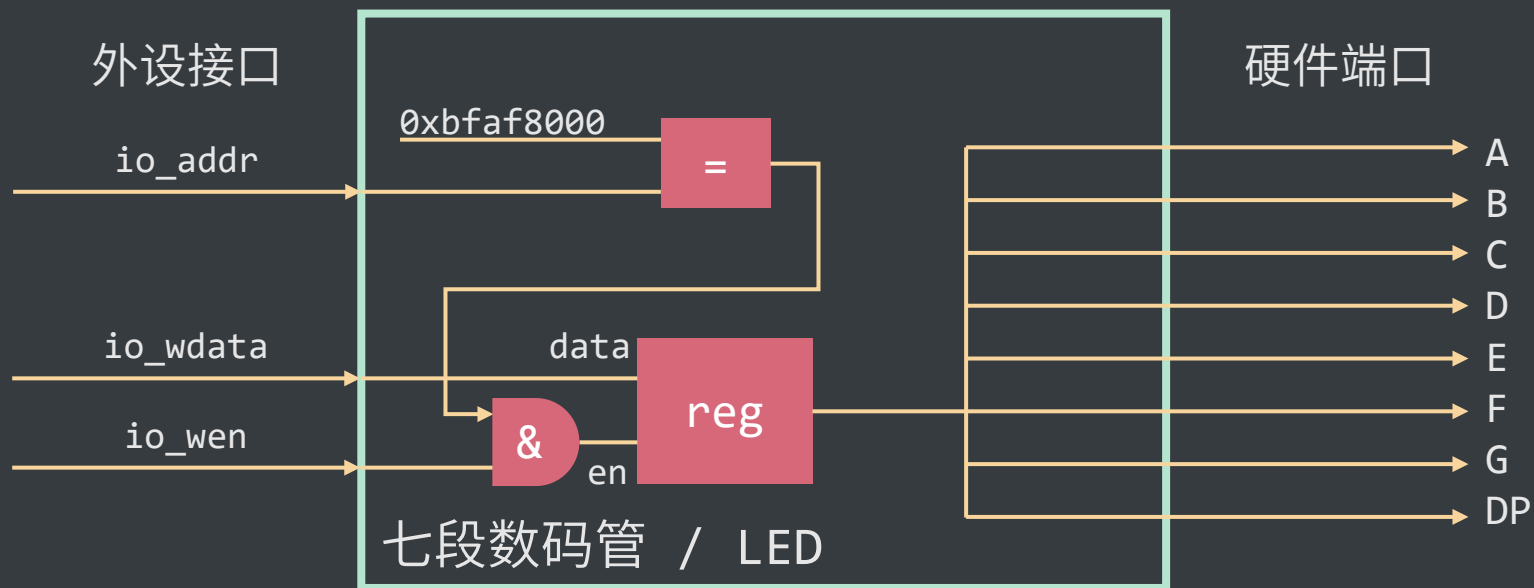
目标



三. 接口与外设

三. 接口与外设

外设模块内部实现



三. 接口与外设

外设模块设计与实现

进阶外设模块内部实现

🏆 挑战

- 参见开发板用户手册
- 串口 (CP2102)、VGA 显示接口、以太网接口 (CH395Q)、SRAM 内存芯片 (IS61WV12816BLL)、蓝牙模块 (BLE-CC41-A)、LCD 显示屏 (JLX128218G-81202)、蜂鸣器

三. 接口与外设

外设测试程序编写与仿真调试

外设测试程序编写



目标



优质实践

- 由于未实现中断、DMA等，故仅能使用轮询方式进行外设通讯
- 输出设备测试程序：实现对应的 SW 指令即可
- 输入设备测试程序：实现对应的 LW 指令即可
- 为了实现轮询，需要编写循环程序
- 为了实现有效输出，需要编写延时等待程序，见下页

三. 接口与外设

外设测试程序编写与仿真调试

外设测试程序编写



优质实践

- 延时等待程序：
 - lui t1, 0x00a0
 - ori t1, 0x0000 // 0x00a00000, approximately 1s@50MHz
 - lui t2, 0xffff
 - ori t2, 0xffff // 0xffffffff = -1
 - loop:
 - add t1, t1, t2
 - beq t1, 0(t0), <next>
 - nop
 - j <loop>
 - nop
 - next:

三. 接口与外设

成果

- 简单的外设模块和接口
- 针对外设的测试程序

四. 从单周期到流水线

- 流水线架构设计
- 流水线元件设计

四. 从单周期到流水线

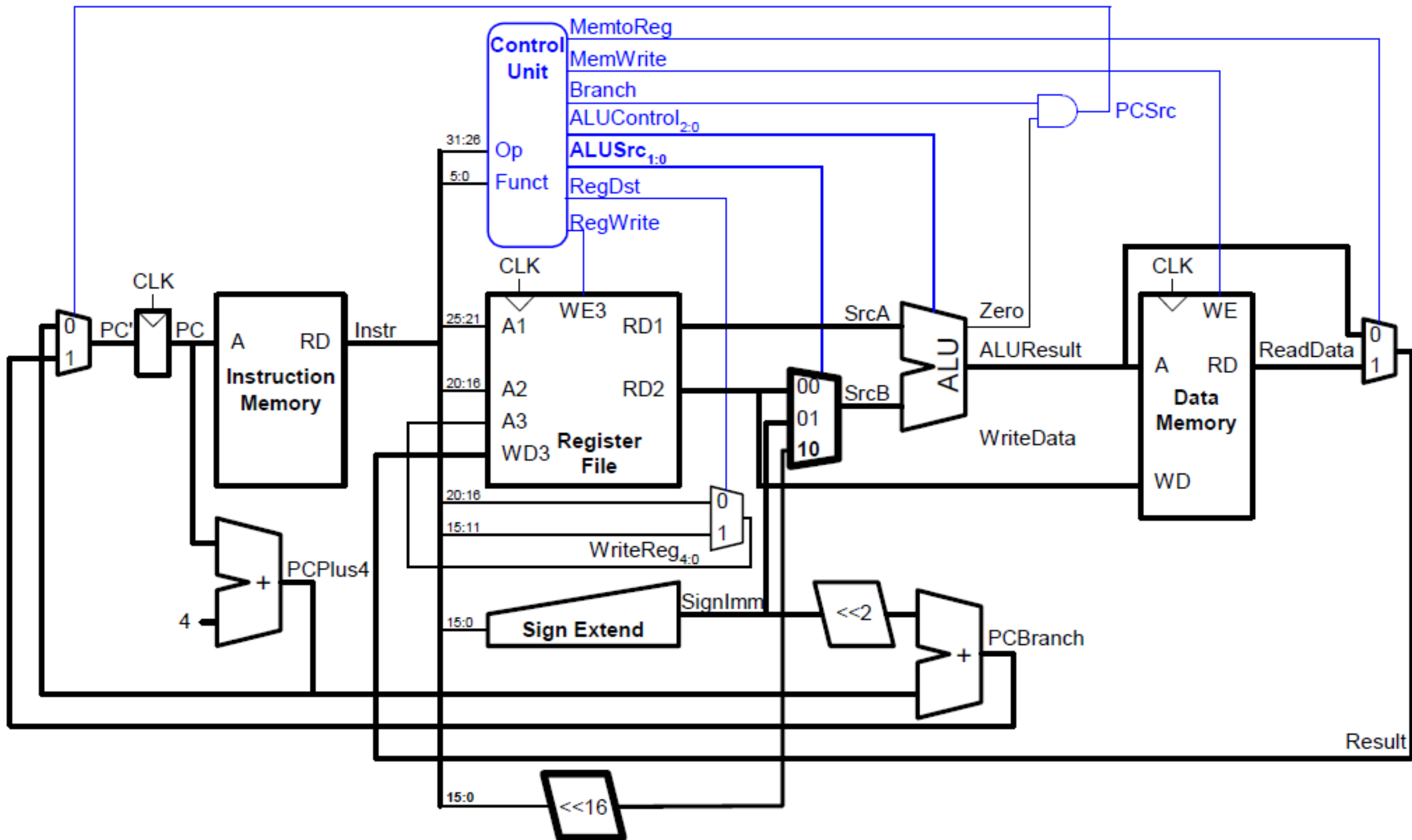
流水线架构设计

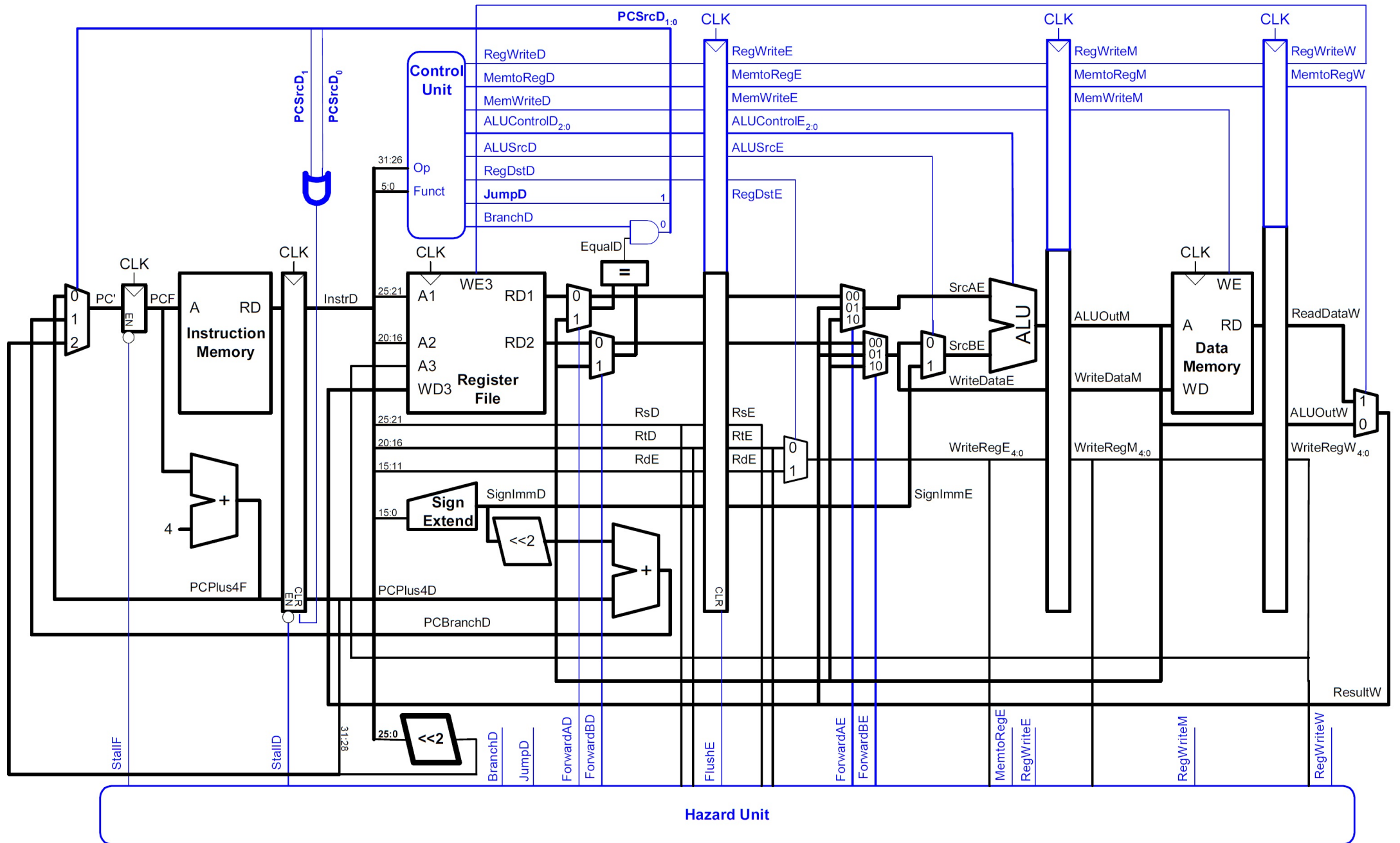
流水线架构特点

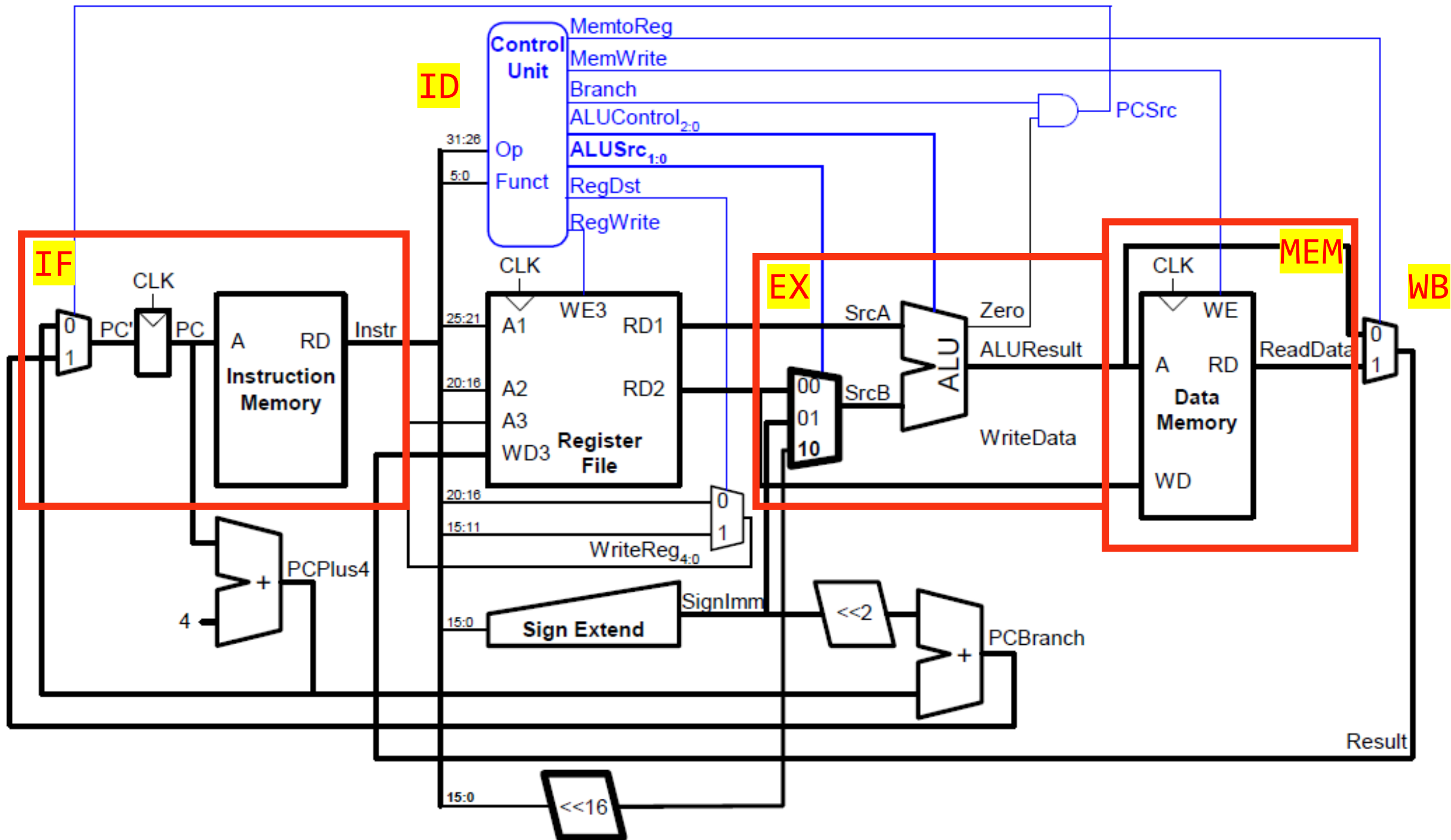


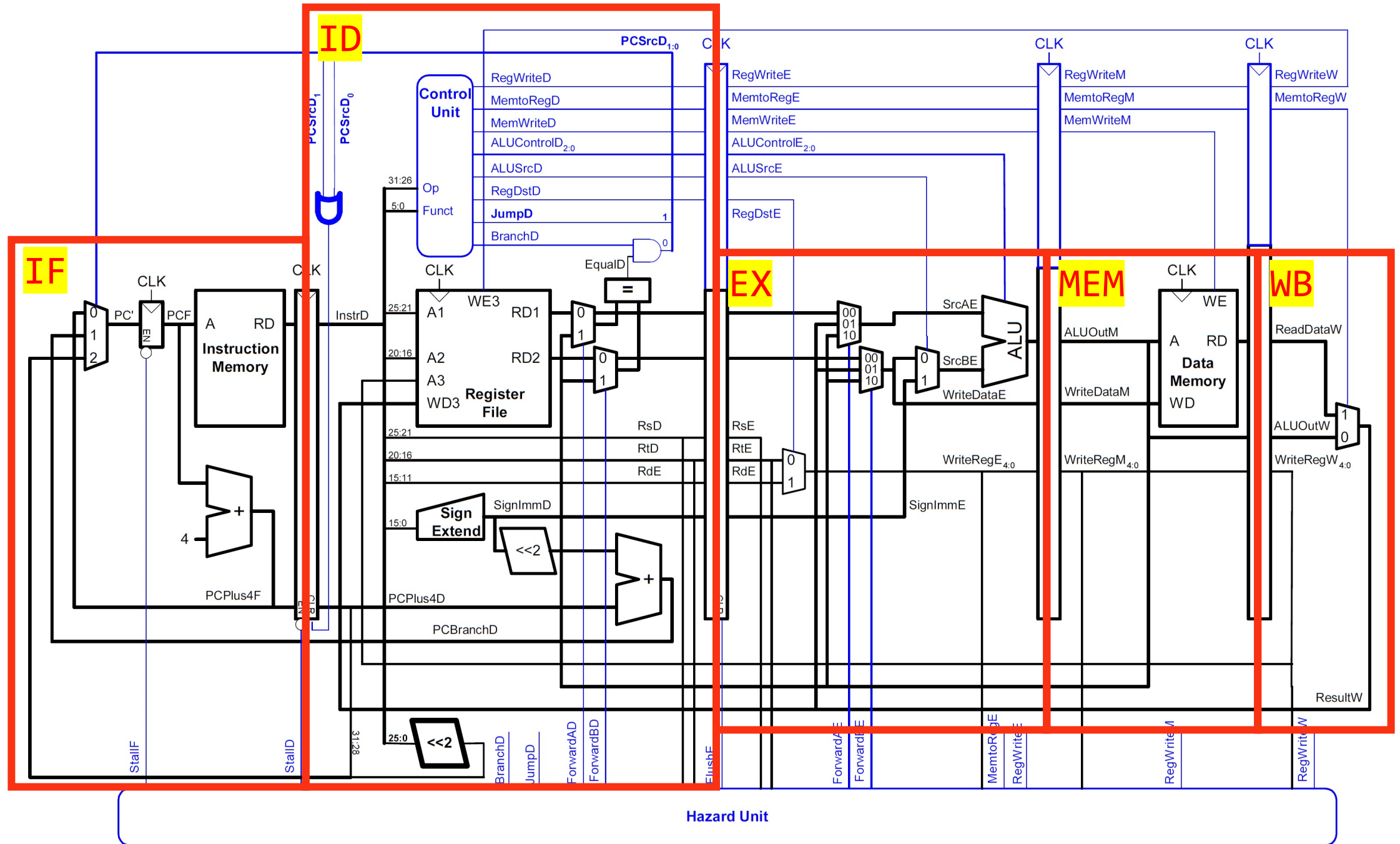
目标

- 流水线架构将不同功能段的信号利用流水级缓冲器隔离，从而实现不同功能段在同一时刻执行不同的指令，并将执行好的指令逐级传递下去
- 最直观来看，在单周期架构的五个功能段之间插入流水级缓冲器，即可得到一个“流水线架构”









四. 从单周期到流水线

流水线架构设计

流水线架构设计



目标

- 在流水线架构中，需要对元件产生的信号进行分析，判断信号的生存周期，并据此确定信号逐级下传的设计
- 为了实现不同流水级之间的隔离，同时满足流水线的时序需求，必须保证：
 - 任一信号必须沿流水线方向逐级经过流水级缓冲器下传
 - 不同级的相同信号不能混用
 - 不同级的信号不能同时引入同一元件中存在逻辑相关的引脚上
 - 同一级内的信号在下一个时钟上升沿到来之前必须达到稳定

四. 从单周期到流水线

流水线架构设计

进阶流水线架构设计

挑战

- 非五级流水架构设计：如“取值、译码、执行”三级流水等
- 非静态流水架构设计
- 超标量/超流水架构设计

四. 从单周期到流水线

流水线元件设计

流水线元件设计

目标

- 流水线架构仅有一个特有的主要元件——流水级缓冲器
- 流水级缓冲器的实现相对简单，仅需在时钟上升沿时将上一级输入的信号传递至下一级，其余时间输出到下一级的信号由内部的 `reg` 保持
- 另外，流水线架构中，`RegFile` 等元件的实现需要进行一些调整，如：
 - `RegFile` 读端口的几个信号需要都为解码段信号
 - `RegFile` 写端口的几个信号需要都为回写段信号
 - `PC` 相关的信号（尤其是 `PC'` 信号）的计算涉及到不同级的信号

四. 从单周期到流水线

流水线元件设计

流水线级的不同实现方式



优质实践

- “课程 10：MIPS 流水线 CPU 设计”、《自己动手写 CPU》中给出的具体实现方法均为将每一级单独封装为一个模块，然后将相应模块所需的所有输入输出连接至模块上
- 这样的实现方法并无问题，但在不注意的情况下，控制路径和数据路径的区分并不清晰，尤其是指令信号等同时包含了控制信息和数据信息的信号，增加了信号设计和检查的难度
- 在我们的项目中，控制信号和数据信号被完全分开连接，而非混合在一起，且所有跨流水级的信号均为单一的数据或控制信号，在生存周期到达之后即不再被下传，相对较好地解决了此问题

四. 从单周期到流水线

成果

- 一个不带冲突处理的经典五级流水架构 CPU
- （基本上没法用）

五. 流水线架构的完善

- 流水线冲突分析和设计
- 流水线测试程序编写与仿真调试

流水线冲突分析

| 目标

- 到目前为止，我们实现的流水线 CPU 无法使用，原因是未对冲突进行相应的处理
- 流水线上的主要冲突：
 - 数据相关：主要为针对同一寄存器的先写后读
 - 阻塞操作：跳转、除法、中断、异常之类需要停止流水线运行的操作

五. 流水线架构的完善

流水线冲突分析和设计

流水线冲突设计

目标

- 主要的冲突处理方式：
 - 重定向：将位于后方流水级的数据信号重定向至前方的流水级，从而让后进入流水线的指令提前获取到已经计算完成的寄存器数据，而无需等待数据被回写
 - 阻塞：对于无法靠重定向解决的数据相关以及阻塞操作，停止流水线的前方一部分流水级（插入空泡），等待所需的数据产生或阻塞操作执行完毕后再继续流水线的运行

五. 流水线架构的完善

流水线冲突分析和设计

冲突单元的设计与实现

目标

- 在冲突分析和设计完毕之后，需要添加一个专门的元件用来检测冲突情况，并根据情况控制相应的冲突信号，如重定向信号、阻塞信号和清空信号等
- 冲突单元不属于任何一个流水级，且冲突单元会从所有的流水级获取需要的控制信号和数据信号，并直接控制每个流水级缓冲器的使能和清空信号
- 冲突单元的实现以组合逻辑为主
- 阻塞某一流水级时，要注意如下两个情况：
 - 在下一周期清空被阻塞的流水级之后的那一级，也即在此处插入一个空泡，具体方法是置流水级缓冲器的清空引脚有效
 - 在当前被阻塞的流水级之前的所有流水级也需要一并被阻塞

五. 流水线架构的完善

流水线冲突分析和设计

面向 Excel 编程 ☺



优质实践

- 指令集.xlsx

五. 流水线架构的完善

流水线测试程序编写与仿真调试

流水线测试程序编写



目标



优质实践

- 针对流水线的测试程序是针对单周期的测试程序的超集
- 除了对每条指令本身的功能进行测试之外，还需要对多条指令之间的各种冲突情况进行测试
- 先设计指令功能测试，此时可以通过添加多余指令或使用不同的寄存器来避免发生相关，从而仅针对指令功能进行测试
- 然后设计冲突测试，此时即需要通过结合之前对冲突的分析，设计不同的相关、阻塞等，尽可能覆盖所有可能的冲突情况

流水线仿真调试

挑战

- 流水线的仿真调试和单周期的调试相比，除了需要考察的信号变多之外，并无本质区别
- 因此，trace 机制仍然可用，但是，在使用 trace 机制时，要注意：
trace 机制比对的 PC、回写情况等应来自回写级，故需要将 PC 一路传至回写级

五. 流水线架构的完善

成果

- 一个能正常工作的经典五级流水架构 CPU

六. 走得更远

- 乘除法器
- 中断与异常
- Cache

六. 走得更远

乘除法器

乘除法器

挑战

- 在 Vivado 中，乘法的实现相对简单，直接使用乘法即可调用 Xilinx 乘法 IP 核，能在一个周期内完成 32 位乘法计算
- 但直接调用除法生成的电路为基于 LUT 的除法器，无论是从性能看还是从占用片内资源看都不尽人意
- 使用 Xilinx 除法器 IP 核，即可使用片内的 DSP 资源完成除法计算，不仅节约了片内资源，更极大增加了除法的计算效率

六. 走得更远

乘除法器

HiLo 寄存器

挑战

- 在实现了乘除法器功能之后，为了真正能执行乘除法指令，还需要实现用于保存乘除法运算结果的 HiLo 寄存器，以及访问 HiLo 寄存器的相关指令（MFHI MFLO MTHI MTLO），关于 HiLo 寄存器的架构和实现请参考 MIPS 架构的相关文档
- 值得注意的是，添加了 HiLo 寄存器实现之后，不仅需要添加相应的控制和数据路径，还需要考虑 HiLo 寄存器的冲突

六. 走得更远

中断与异常

中断与异常

挑战

- MIPS 的中断和异常机制较为复杂，为了实现中断和异常功能，需要实现 CP0 寄存器，具体的实现方法请参考“课程 11：MIPS 中断与异常原理与设计”，以及 MIPS 架构相关文档

六. 走得更远

Cache

Cache



挑战

- Cache 的设计与实现，请参考“课程 12：MIPS 存储系统原理以及 Cache 模块设计与实现”

六. 走得更远

成果

- 一个真实可用的 CPU

七. 结语

多说几句

- 时间紧：建议一周以内解决工具链问题并完成单周期及测试，最好能直接走通整个工作流，之后两周时间完成流水线和接口的设计实现以及调试
- 任务重：建议组内保证良好沟通，缺乏良好沟通的合作对于系统性极高的本项目来说是致命的
- 分工难：建议组内能力较强的同学牵头进行架构开发和接口设计，其余同学可以并行开展元件内部实现设计、接口元件实现、测试程序编写等任务，在元件开发时注意单元测试，最后合作进行整体调试
- 路径明确：架构设计坑很多，按现有的技术路径走能避开不少问题，在涉及架构涉及时尽量参考现有的实现而非从零开始造轮子

谢谢！

MIPS：入手与「优质」实践

MIPS: A Taste & Good Practice

陈康冰 邓飞 贾泽琛 谢东桓